



DZPHARM · ARCHITECTURE · FOUNDATION

State Management

A comprehensive technical design document for DzPharm's state management architecture: the Zustand store, the 6 state gaps (no Copilot history, interactions list lost on refresh, recently-viewed capped at 1, no cross-tab sync, active view in component state, medication detail in modal state), the "same-day three" fixes, URL-state layer, cross-tab sync via BroadcastChannel, and server-side state after #8 Auth.

6

STATE GAPS

v2

STORE SCHEMA

8

FEATURES AFFECTED

~2w

FULL UPGRADE

DEEP-DIVE OF

Feature #2 – State Management

Document Structure

This document is a comprehensive technical design for upgrading DzPharm's state management from C+ grade (6 state gaps, features losing data on refresh) to A+ grade (persistent, URL-backed, cross-tab synced, server-side after auth). It is organized in 10 chapters covering the current Zustand architecture, the 6 state gaps, the URL-state layer, cross-tab sync, the "same-day three" fixes, server-side state, testing strategy, and the final roadmap.

01	The State Problem · 6 Gaps	p. 3
02	The Zustand Store Architecture	p. 7
03	Gap #1 · Active View in Component State	p. 11
04	Gap #2 · Medication Detail in Modal State	p. 14
05	Gap #3 · Interactions List Not Persisted	p. 17
06	Gap #4 · No Copilot History	p. 20
07	The Same-Day Three (1-Day Fixes)	p. 23
08	Cross-Tab Sync & Server-Side State	p. 26
09	Testing & Migration Strategy	p. 29
10	Final Roadmap & Verdict	p. 32

6 Gaps · Features Losing Data on Refresh

State management is the platform's **third foundation** — after #1 Routing (URLs) and #8 Auth (identity). The Zustand store is well-architected (schema versioning, typed fields, separate theme key), but 6 state gaps cause real user frustration. Each gap represents a feature that «works while you're looking at it, then vanishes when you refresh».

Nomenclature officielle — Juin 2026

L'intelligence pharmaceutique algérienne

Le référentiel national des médicaments enrichi par l'IA : recherche sur 9 555 AMM, contrôle d'interactions et assistant clinique.

24 livres pharmacologie

932 monographies

1 791 prix PPA

RCP ANSM officiel

Grossesse CRAT

PÉRIMÈTRE : TousDCI / MoléculeMarqueLaboratoireN° AMM

Rechercher par dosage & forme (ex: Amox 500mg sirop, Augmentin 1g)ScannerEntrée

Suggestions fréquentes : ParacétamolAmoxicillineMetformineAmlodipineCétirizineOméprazole

Accès rapide

Sélection adaptée au mode professionnel

Répertoire
Rechercher un médicament par marque, DCI, laboratoire ou n° AMM.

Contrôle d'interactions
Vérifier les associations médicamenteuses avant dispensation.

RCP & monographies
911 monographies DCI issues des livres techniques de pharmacologie.

Calculs cliniques
Posologies pédiatriques, fonction rénale, grossesse, Chifa, Ramadan.

Copilote IA
Assistant clinique en français, arabe et darija.

MÉDICAMENTS ACTIFS
5 381
AMM en cours de validité

PRODUCTION LOCALE
3 840 71%
Part des actifs fabriqués en Algérie

IMPORTATIONS
1 541
Actifs importés

RETIRÉS DU MARCHÉ
2 679
Historique des retraits

Les plus consultés

Classement DzPharm — consultations cumulées

1

DOLIPRANE
PARACETAMOL

7 consultations

2

ACARLYSE
ACARBOSE

5 consultations

3

CLAVODAR FORTE 400/57
AMOXICILLINE (SOUS FORME TRIHYDRATEE)/ACI...

4 consultations

4

ABAVIR OS
ABACAVIR SULFATE EXPERIME EN ABACAVIR

3 consultations

5

BIOPAMOX
AMOXICILLINE SODIQUE

2 consultations

6

CO-DOLYC
PARACETAMOL/CODEINE PHOSPHATE HEMIHYD...

2 consultations

7

IMETH
METHOTREXATE

2 consultations

8

LEMBLASTINE
VINBLASTINE (SOUS FORME DE SULFATE)

2 consultations

DCI les plus recherchées

Classement par principe actif — consultations cumulées

1

PARACETAMOL
DOLIPRANE

9

2

ACARBOSE
ACARLYSE

5

3

ABACAVIR SULFATE EXPE...
ABAVIR OS - ABAVIR

4

4

AMOXICILLINE (SOUS FOR...
CLAVODAR FORTE 400/57

4

5

PARACETAMOL/CODEINE ...
CO-DOLYC - COPARALGAN

3

6

AMOXICILLINE SODIQUE
BIOPAMOX

2

7

METHOTREXATE
IMETH

2

8

VINBLASTINE (SOUS FOR...
LEMBLASTINE

2

Tous les outils

Aides à la dispensation adaptées aux spécificités algériennes

Posologies pédiatriques

Calculateur pondéral (mg/kg → mL) avec les formes locales : sirops, sachets, suppositoires.

Simulateur Chifa

Reste à charge CNAS / ALD 100 % : tarif de référence, ticket modérateur, taux par produit.

Adaptateur Ramadan

Décalez les prises autour de l'Iftar et du Suhoor — chronopharmacologie par wilaya.

Compareur

Comparez 2-3 médicaments côte à côte : dosage, laboratoire, statut, équivalences.

Catalogue & prix

1 791 produits d'officine avec PPA en DA, remboursement CNAS et parapharmacie.

Contrôle d'interactions

Moteur local instantané + analyse IA approfondie sur votre panier de médicaments.

Armoire familiale

Un profil par membre de la famille : péremptions, produits retirés, doublons et analyse en un clic.

Copilote IA

Assistant clinique FR / arabe / darija, modes professionnel et patient.

Fonction rénale

Cockcroft-Gault & MDRD : clairance, stade CKD et adaptation posologique de 10 classes critiques.

Figure 2.1 – Dashboard home. The Zustand store at `localStorage["dzpharm-store"]` holds: `designMode`, `palette`, `audience`, `favorites`, `recentlyViewed`, `armoireMembers`, `armoireEntries`, `armoireJournal`, `chifaCardType`, `chifaLines`. But 6 critical pieces of state are NOT in the store – they're in component state or not persisted at all.

1.1 The 6 State Gaps

#	GAP	WHERE STATE LIVES	WHAT HAPPENS ON REFRESH	AFFECTED FEATURES
1	Active view in component state	<code>useState</code> in header	Drops to dashboard home	#1 (Routing), all views
2	Medication detail in modal state	<code>useState</code> in Répertoire	Modal closes, detail lost	#18 (Medication Detail)
3	Interactions drug list not persisted	<code>useState</code> in Interactions	Full drug list wiped (3-5 min lost)	#22 (Interactions Checker)
4	No Copilot conversation history	Not stored anywhere	Conversation lost, re-ask = \$\$	#36 (Copilot), #40 (History)
5	Recently-viewed capped at 1 entry	Zustand (but buggy cap)	Only last-viewed survives	#44 (Recently Viewed)
6	No cross-tab synchronization	Per-tab Zustand instances	Tab A's favorites invisible in Tab B	#48 (Favorites), #43 (Notifications)

1.2 The Pattern · «Works While Looking, Vanishes on Refresh»

All 6 gaps share the same pattern: the state works correctly while the user is on the page, but is lost on refresh, tab close, or navigation away. This is because the state lives in React component `useState` (which is ephemeral) rather than in the Zustand store (which persists) or in the URL (which is shareable). The fix for each gap is to move the state from `useState` to either the Zustand store or the URL — depending on whether the state should be persisted (Zustand) or shareable (URL).

1.3 The State Decision Matrix

Not all state should live in the same place. The decision matrix below guides where each piece of state should live based on its requirements:

STATE TYPE	PERSIST?	SHARE?	SYNC ACROSS TABS?	WHERE IT SHOULD LIVE
Active view (which page)	Yes	Yes	No	URL (the route itself)
Search query + filters	Yes	Yes	No	URL search params (?q=doliprane)
Drug interactions list	Yes	Yes	No	URL search params (?drugs=8421,1523)
Copilot conversation	Yes	No	Yes	Zustand (Phase 1) → Server (Phase 2)
Favorites	Yes	No	Yes	Zustand (Phase 1) → Server (Phase 2)
Recently viewed	Yes	No	Yes	Zustand (Phase 1) → Server (Phase 2)
Theme + palette	Yes	No	Yes	Zustand (already correct)
Medication detail modal	Yes	Yes	No	URL (intercepting route, #1)

THE FOUNDATIONAL GAP

State management is the third foundation — after routing and auth. Every feature that «loses data on refresh» (#22 interactions, #40 Copilot history, #44 recently-viewed, #48 favorites) traces back to #2. Fixing #2 is a precondition for these features reaching A-grade. The "same-day three" fixes (Chapter 7) close 3 of the 6 gaps in a single day with ~150 lines of code.

What Works & What's Missing

DzPharm uses **Zustand with persist middleware** for its client state. This is an excellent choice — Zustand is the lightest-weight React state library (~1KB), its persist middleware handles localStorage serialization automatically, and its schema-versioning system enables safe migrations when the state shape changes. The store is well-architected but incomplete: 4 critical pieces of state are missing from it.

2.1 The Current Store Schema (Version 2)

```
// localStorage.getItem("dzpharm-store") - parsed
{
  "state": {
    "designMode": "standard",          // "standard" | "compact" | "accessibility"
    "palette": "porcelain",           // 20 palette names from the Nuancier
    "audience": "pro",               // "pro" | "family" | "botanical"
    "favorites": [                    // Array of {id, brand, dci, addedAt}
      {{ "id": 9775, "brand": "DOLIPRANE", "dci": "PARACETAMOL", "addedAt": 1789821578925 }}
    ],
    "recentlyViewed": [               // Capped at 1 entry (Gap #5)
      {{ "id": 9775, "brand": "DOLIPRANE", "dci": "PARACETAMOL", "status": "ACTIF", "viewedAt": 1789821578925 }}
    ],
    "armoireMembers": [],             // Family cabinet members
    "armoireEntries": [],            // Per-member medication entries
    "armoireJournal": [],            // Event log (add/remove/expire)
    "chifaCardType": "standard",     // "standard" | "ald" | "chronique"
    "chifaLines": []                 // Items in the Chifa reimbursement simulator
  },
  "version": 2                       // Schema version - enables migrations
}
```

2.2 What Works Well

Schema versioning (version: 2)

The `"version": 2` field is Zustand's built-in migration mechanism. If the state shape changes, the persist middleware can run a `migrate` function to upgrade older snapshots. This is the right pattern and rare in production apps.

Type-stable field design

Each entry uses stable typed fields: `id` (number), `brand` (string), `addedAt` (epoch ms). No nested objects, no arrays of arrays. This makes serialization bullet-proof and migration trivial.

Separate theme key

The theme is stored OUTSIDE the Zustand store (`localStorage["theme"]`). This is correct — the theme must be readable by an inline script in `<head>` before React hydrates, to avoid FOUC (flash-of-unstyled-content).

2.3 What's Missing

No Copilot conversation history

The store has no `copilotHistory` key. Every Copilot conversation is lost on refresh. For a Gemini-powered assistant that costs money per call, this is both a UX and a cost issue — users re-ask the same question, each costing the platform money.

No interactions drug list

The store has no `interactionDrugs` key. The drug list in the Interactions Checker lives in `useState`. A pharmacist who builds a 6-drug interaction list and accidentally refreshes loses everything — 3-5 minutes of work.

recentlyViewed capped at 1 entry

The `recentlyViewed` array contains only 1 entry after viewing multiple medications. This is either an intentional cap (should be 10) or a bug in the dedup logic (cap should be enforced after dedup, not before).

No cross-tab synchronization

The Zustand store is per-tab (each tab gets its own instance). Favorites added in Tab A are not visible in Tab B until Tab B refreshes. This is a localStorage limitation — the `storage` event can fix it (Chapter 8).

2.4 The Target Store Schema (Version 3)

```
// Target schema - version 3 (adds copilotHistory + interactionDrugs)
{
  "state": {
    // — Existing (keep as-is) —
    "designMode": "standard",
    "palette": "porcelain",
    "audience": "pro",
    "favorites": [...],
    "recentlyViewed": [...], // Fixed: cap raised to 10
    "armoireMembers": [],
    "armoireEntries": [],
    "armoireJournal": [],
    "chifaCardType": "standard",
    "chifaLines": [],

    // — New (Phase 1 - same-day three) —
    "copilotMessages": [], // Copilot conversation (Gap #4)
    "copilotMode": "pro", // Copilot mode persistence (#37)

    // — New (Phase 2 - after #1 Routing) —
    // interactionDrugs moves to URL params, not Zustand
    // URL: /interactions?drugs=8421,1523&ctx=Patient+72+ans
  },
  "version": 3 // Schema upgrade - migrate v2 → v3
}
```

2.5 The Migration Function

```
// The Zustand persist migration function
const persistedStore = persist(
  (set, get) => ({ /* ... */ }),
  ({
    name: 'dzpharm-store',
    version: 3, // Current version
    migrate: (persistedState, version) => ({
      const state = persistedState as StoreState;

      // v2 → v3: add copilotMessages + copilotMode
      if (version < 3) {{
        state.copilotMessages = [];
        state.copilotMode = 'pro';
      }}

      // v1 → v2 (already done): added armoire, chifa
      if (version < 2) {{
        state.armoireMembers = [];
        state.armoireEntries = [];
        state.chifaLines = [];
      }}

      return state;
    }},
    // Prune: keep last 30 Copilot messages, last 10 recently-viewed
    partialize: (state) => ({
      ...state,
      copilotMessages: state.copilotMessages.slice(-30),
      recentlyViewed: state.recentlyViewed.slice(0, 10),
    })),
  })
);
```

ZUSTAND IS THE RIGHT CHOICE

Zustand with persist is the correct state management choice for DzPharm. It's ~1KB (vs Redux's 15KB), its persist middleware handles localStorage automatically, and its schema-versioning system (the `version` + `migrate` pattern) is rare in production apps. The store is well-architected — it just needs 4 more keys added and 2 bugs fixed (recentlyViewed cap, cross-tab sync).

— CHAPTER 03 · GAP #1 · ACTIVE VIEW IN COMPONENT STATE

setState Navigation · The Root of #1

Gap #1 is the root cause of #1 Routing Architecture. The active view (which page the user is on) is tracked in React `useState` in the header component — not in the URL. Clicking «Répertoire» calls `setActiveView('repertoire')`, which conditionally renders the Répertoire component, but the URL stays at `/`. Refresh drops the user to dashboard home.

Nomenclature officielle — Juin 2026

L'intelligence pharmaceutique algérienne

Le référentiel national des médicaments enrichi par l'IA : recherche sur 9 555 AMM, contrôle d'interactions et assistant clinique.

24 livres pharmacologie

932 monographies

1 791 prix PPA

RCP ANSM officiel

Grossesse CRAT

PÉRIMÈTRE : Tous DCI / Molécule Marque Laboratoire N° AMM

Rechercher par DCI (ex: Paracétamol, Amoxicilline...)

Scanner

Entrée

Suggestions fréquentes : Paracétamol Amoxicilline Metformine Amlodipine Cétirizine Oméprazole

Accès rapide

Sélection adaptée au mode professionnel

Répertoire

Rechercher un médicament par marque, DCI, laboratoire ou n° AMM.

Contrôle d'interactions

Vérifier les associations médicamenteuses avant dispensation.

RCP & monographies

911 monographies DCI issues des livres techniques de pharmacologie.

Calculs cliniques

Posologies pédiatriques, fonction rénale, grossesse, Chifa, Ramadan.

Copilote IA

Assistant clinique en français, arabe et darja.

MÉDICAMENTS ACTIFS

5 381

AMM en cours de validité

PRODUCTION LOCALE

3 840 71%

Part des actifs fabriqués en Algérie

IMPORTATIONS

1 541

Actifs importés

RETIRÉS DU MARCHÉ

2 679

Historique des retraits

Les plus consultés

Classement DzPharm — consultations cumulées

1 DOLIPRANE

PARACETAMOL

7 consultations

2 ACARLYSE

ACARBOSE

5 consultations

3 CLAVODAR FORTE 400/57

AMOXICILLINE (SOUS FORME TRIHYDRATEE)/ACI...

4 consultations

4 ABAVIR OS

ABACAVIR SULFATE EXPERIME EN ABACAVIR

3 consultations

5 BIOPAMOX

AMOXICILLINE SODIQUE

2 consultations

6 CO-DOLYC

PARACETAMOL/CODEINE PHOSPHATE HEMIHYD...

2 consultations

7 IMETH

METHOTREXATE

2 consultations

8 LEMBLASTINE

VINBLASTINE (SOUS FORME DE SULFATE)

2 consultations

DCI les plus recherchées

Classement par principe actif — consultations cumulées

1 PARACETAMOL

DOLIPRANE

9

2 ACARBOSE

ACARLYSE

5

3 ABACAVIR SULFATE EXPE...

ABAVIR OS - ABAVIR

4

4 AMOXICILLINE (SOUS FOR...

CLAVODAR FORTE 400/57

4

5 PARACETAMOL/CODEINE ...

CO-DOLYC - COPARALGAN

3

6 AMOXICILLINE SODIQUE

BIOPAMOX

2

7 METHOTREXATE

IMETH

2

8 VINBLASTINE (SOUS FOR...

LEMBLASTINE

2

Tous les outils

Aides à la dispensation adaptées aux spécificités algériennes

Posologies pédiatriques

Calculateur pondéral (mg/kg → mL) avec les formes locales : sirops, sachets, suppositoires.

Simulateur Chifa

Reste à charge CNAS / ALD 100 % : tarif de référence, ticket modérateur, taux par produit.

Adaptateur Ramadan

Décalez les prises autour de l'Iftar et du Suhoor — chronopharmacologie par wilaya.

Compareur

Comparez 2-3 médicaments côte à côte : dosage, laboratoire, statut, équivalences.

Catalogue & prix

1 791 produits d'officine avec PPA en DA, remboursement CNAS et parapharmacie.

Contrôle d'interactions

Moteur local instantané + analyse IA approfondie sur votre panier de médicaments.

Armoire familiale

Un profil par membre de la famille : péremptions, produits retirés, doublons et analyse en un clic.

Copilote IA

Assistant clinique FR / arabe / darja, modes professionnel et patient.

Fonction rénale

Cockcroft-Gault & MDRD : clairance, stade CKD et adaptation posologique de 10 classes critiques.

Figure 2.2 – Search via URL. After the routing migration (#1), searching «doliprane» updates the URL to `/repertoire?q=doliprane`. The search state is now in the URL – shareable, back-button-able, refresh-survivable. But without #1, the search state is in `useState` and lost on refresh.

3.1 The Current Pattern

```
// Current - useState navigation (broken)
const [activeView, setActiveView] = useState('dashboard');

const handleNav = (view) => {{
  setActiveView(view); // URL stays at /
}};

// Conditional rendering based on activeView
{{activeView === 'repertoire' && <Repertoire />}}
{{activeView === 'interactions' && <Interactions />}}
{{activeView === 'copilote' && <Copilot />}}
```

3.2 The Fix · URL Routing (#1 Dependency)

The fix for Gap #1 is the routing migration covered in the #1 Routing Architecture Technical Design Document. The active view moves from `useState` to the URL itself — each view gets a real route (`/repertoire`, `/interactions`, `/copilote`). The `useState` pattern is replaced with `next/link` anchors and `usePathname()` for active state. Gap #1 is fully closed by #1 Phase 1 (3 days).

3.3 Why This Gap Is #1's Responsibility

Gap #1 and #1 Routing are the same problem viewed from different angles. From the state-management perspective: «the active view is in `useState`, not in the URL». From the routing perspective: «there are no real routes, navigation uses `setState`». Both perspectives converge on the same fix: implement Next.js App Router with real URLs. This is why #2 depends on #1 — the state management cannot be fixed without the routing foundation.

— CHAPTER 04 · GAP #2 · MEDICATION DETAIL IN MODAL STATE

The Modal-Without-URL Problem

Gap #2 is the medication detail modal. Clicking a medication in the Répertoire opens a modal overlay — but the URL doesn't change. The medication detail cannot be deep-linked, shared, or bookmarked. Refresh closes the modal and returns to the Répertoire list. The fix is the intercepting route pattern covered in #1 Chapter 6.

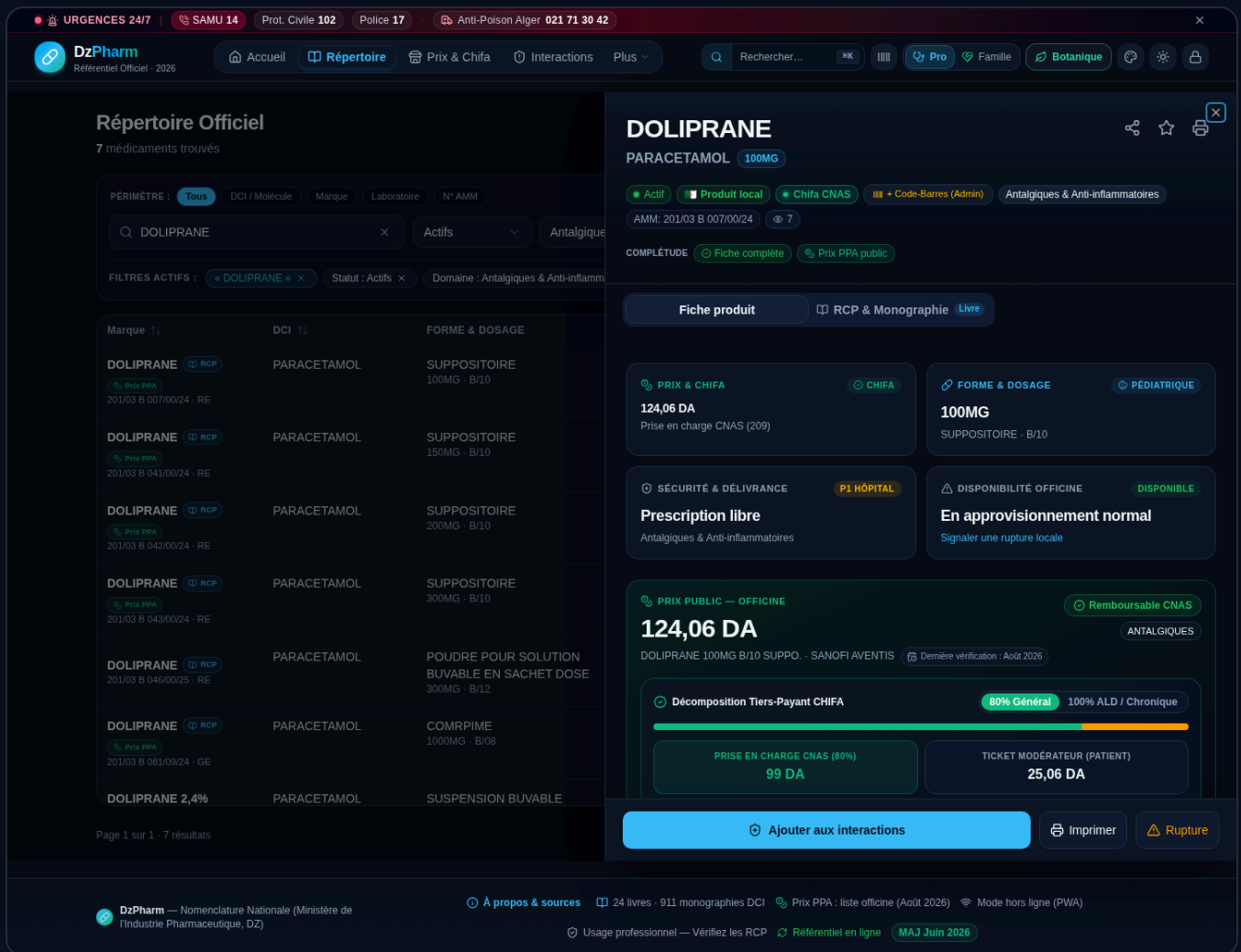


Figure 2.3 – Medication detail modal. The URL doesn't change when the modal opens. With the intercepting route pattern (#1 Chapter 6), the URL becomes `/medicament/doliprane-500mg-comprime` – shareable, bookmarkable, refresh-survivable.

4.1 The Current Pattern

```
// Current – modal state in useState (broken)
const [selectedMed, setSelectedMed] = useState(null);

// Clicking a medication opens the modal
<button onClick={() => setSelectedMed(med)}>
  {{med.brand}}
</button>

// Modal renders conditionally – URL doesn't change
{{selectedMed && (
  <MedicationModal med={{selectedMed}} onClose={() => setSelectedMed(null)} />
)}}
```

4.2 The Fix · Intercepting Routes (#1 Dependency)

The fix is the intercepting route pattern from #1 Chapter 6. The medication detail gets a real URL (`/medicament/[slug]`) AND can be displayed as a modal when navigated to from the Répertoire. The `useState` modal state is replaced with URL-based navigation. Gap #2 is closed by #1 Phase 3 (3 days, after Phase 1).

4.3 Why Modal State ≠ URL State

The medication detail is a special case: it needs both modal UX (fast overlay from the Répertoire) AND URL state (shareable, refresh-survivable). The intercepting route pattern solves this: client-side navigation from the Répertoire shows the modal; direct URL access shows the full page. Both use the same data-fetching logic, just different layouts. This is the same pattern used by Instagram, Twitter, and GitHub.

— CHAPTER 05 • GAP #3 • INTERACTIONS LIST NOT PERSISTED

The Most Expensive State-Loss on the Platform

Gap #3 is the drug interactions list. When a pharmacist adds 6 drugs to the Interactions Checker, the list lives in `useState` — not in the Zustand store, not in the URL. A single accidental refresh wipes the entire list. Rebuilding a 6-drug list takes 3-5 minutes. This is the **most expensive state-loss on the platform** — and the fix is a single hook.

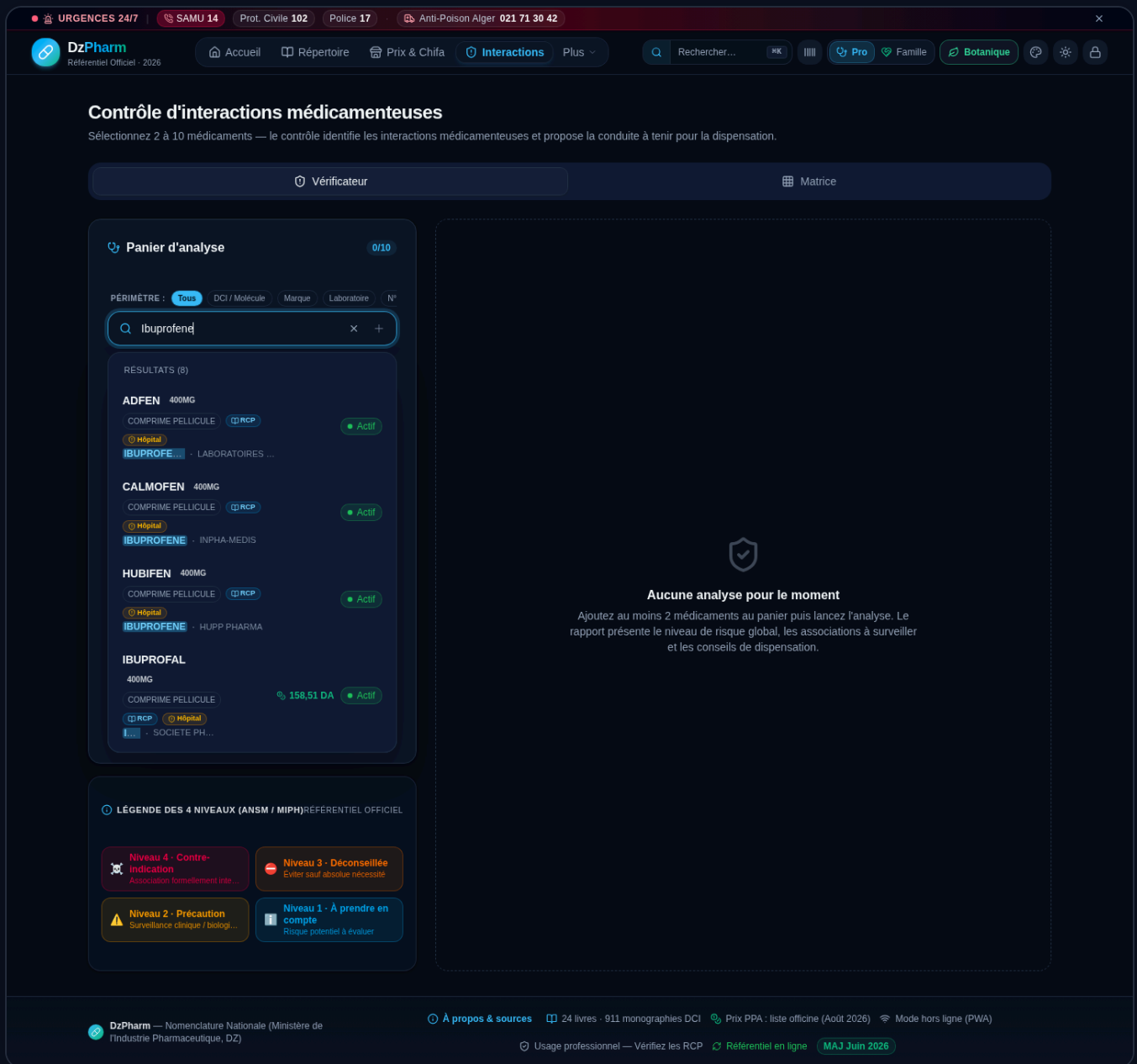


Figure 2.4 — Interactions form with 2 drugs added. The drug list is in `useState`. Neither the URL nor the Zustand store contains any reference to the drugs. Refresh would wipe both entries.

5.1 The Current Pattern

```
// Current - drug list in useState (broken)
const [drugs, setDrugs] = useState([]);
const [patientContext, setPatientContext] = useState('');

// Adding a drug via combobox
const addDrug = (med) => {{
  setDrugs(prev => [...prev, med]); // useState only - lost on refresh
}};

// URL stays at /interactions - no drug state in URL
```

5.2 The Fix · URL Search Params (Same-Day Three, Fix #1)

The fix is the `useInteractionListState` hook — a custom hook that reads/writes the drug list and patient context to URL search params. The URL becomes `/interactions?drugs=8421,1523,9775&ctx=Patient+72+ans+insuffisance+rénale`. The drug list is now: (1) shareable

(copy the URL), (2) back-button-able (previous list restores), (3) refresh-survivable, (4) bookmarkable.
Effort: 4 hours.

```
// lib/useInteractionListState.ts - the fix
import {{ useSearchParams, useRouter, usePathname }} from 'next/navigation';
import {{ useCallback }} from 'react';

export function useInteractionListState() {{
  const searchParams = useSearchParams();
  const router = useRouter();
  const pathname = usePathname();

  // Read drug IDs from URL: ?drugs=8421,1523,9775
  const drugIds = searchParams.get('drugs')
    ?.split(',')
    .map(Number)
    .filter(Boolean) ?? [];

  const patientContext = searchParams.get('ctx') ?? '';

  const setDrugs = useCallback((ids: number[]) => {{
    const params = new URLSearchParams(searchParams);
    if (ids.length > 0) {{
      params.set('drugs', ids.join(','));
    }} else {{
      params.delete('drugs');
    }}
    router.push(`$${{pathname}}?${{params}}`);
  }}, [searchParams, router, pathname]);

  const setPatientContext = useCallback((ctx: string) => {{
    const params = new URLSearchParams(searchParams);
    if (ctx) {{
      params.set('ctx', ctx);
    }} else {{
      params.delete('ctx');
    }}
    router.$${{pathname}}?${{params}}`;
  }}, [searchParams, router, pathname]);

  return {{ drugIds, setDrugs, patientContext, setPatientContext }};
}}
```

5.3 Why URL State (Not Zustand)

The interactions drug list should be in the URL, not in Zustand, because: (1) it's shareable — a pharmacist can send a colleague a link to a specific drug interaction check, (2) it's bookmarkable — a pharmacist can bookmark a common drug combination, (3) it doesn't need cross-tab sync (each tab can have its own interaction check), (4) it doesn't need server-side persistence (it's ephemeral, not personal data like favorites). URL state is the correct choice for shareable, ephemeral state.

Every Conversation Lost on Refresh

Gap #4 is the missing Copilot conversation history. The Zustand store has no `copilotMessages` key. Every conversation exists only in React component state for the duration of the page session. Refresh, tab close, or navigation away from the Copilot view wipes it. For a Gemini-powered assistant that costs money per call, this is both a UX and a cost issue — users re-ask the same question, each costing the platform money.

6.1 The Fix · Zustand Slice (Same-Day Three, Fix #2)

The fix is a `copilotMessages` slice in the Zustand store with persist middleware. On every new message (user or assistant), the store updates. On Copilot mount, the store rehydrates and the previous conversation is restored. Effort: 2 hours (covered in detail in #36 Copilot Chapter 6 and #40 Conversation History).

6.2 The Copilot Zustand Slice

```
// stores/copilotStore.ts - Zustand + persist
import {{ create }} from 'zustand';
import {{ persist }} from 'zustand/middleware';

interface Message {{
  id: string;
  role: 'user' | 'assistant';
  content: string;
  mode: 'pro' | 'patient' | 'enfant';
  timestamp: number;
}}

interface CopilotState {{
  messages: Message[];
  mode: 'pro' | 'patient' | 'enfant';
  addMessage: (msg: Message) => void;
  setMode: (mode) => void;
  clearConversation: () => void;
}}

export const useCopilotStore = create<CopilotState>()(
  persist(
    (set) => ({
      messages: [],
      mode: 'pro',
      addMessage: (msg) => set((s) => ({
        messages: [...s.messages, msg],
      })),
      setMode: (mode) => set({ mode }),
      clearConversation: () => set({ messages: [] }),
    }),
    {{
      name: 'dzpharm-copilot',
      partialize: (state) => ({
        messages: state.messages.slice(-30), // Prune: last 30
        mode: state.mode,
      }),
    }}
  )
);
```

6.3 Why Zustand (Not URL) for Copilot History

The Copilot conversation should be in Zustand, not the URL, because: (1) it's personal — a pharmacist's clinical queries shouldn't be in the URL (visible to anyone looking over their shoulder or in browser history), (2) it's too long — a 10-message conversation would make the URL unwieldy, (3) it doesn't need to be shareable — you don't share a conversation thread via URL (that's what export is for), (4) it needs cross-tab sync — a pharmacist who starts a conversation in one tab should see it in another. Zustand is the correct choice for personal, persistent state.

6.4 The Server-Side Path (After #8 Auth)

Once per-user accounts exist (#8 Auth), the Copilot history moves from Zustand (per-browser) to a server-side `CopilotConversation` table keyed by `userId`. The conversation syncs across devices — a pharmacist who starts a conversation at work can continue it at home. This is covered in detail in #36 Copilot Chapter 6 and #40 Conversation History. The Zustand slice is the Phase 1 fix; the server-side table is the Phase 2 fix.

— CHAPTER 07 · THE SAME-DAY THREE

3 Fixes · 1 Day · ~150 Lines of Code

Three of the six state gaps can be fixed in a single day with roughly 150 lines of code and no architectural changes. These are the «same-day three»: (1) drug list in URL params, (2) Copilot history in Zustand, (3) cross-tab sync via storage event. Together they close 3 of the 6 gaps and lift the grade from C+ to B+.

7.1 Fix #1 · Drug List in URL Params (4 hours)

The `useInteractionListState` hook (Chapter 5.2) moves the interactions drug list from `useState` to URL search params. The URL becomes `/interactions?drugs=8421,1523&ctx=Patient+72+ans`. Closes Gap #3.

7.2 Fix #2 · Copilot History in Zustand (2 hours)

The `copilotMessages` slice (Chapter 6.2) adds Copilot conversation history to the Zustand store with `persist`. Closes Gap #4.

7.3 Fix #3 · Cross-Tab Sync via Storage Event (2 hours)

The `storage` event fires when another tab modifies `localStorage`. By listening to this event and re-hydrating the Zustand store, changes in Tab A (e.g., adding a favorite) are immediately visible in Tab B. Closes Gap #6 (partially — full cross-tab sync requires #8 Auth for session state).

```
// Fix #3 – cross-tab sync via storage event
import {{ useEffect }} from 'react';
import {{ useStore }} from 'zustand';

export function useCrossTabSync() {{
  const rehydrate = useStore(useThemeStore, (s) => s.rehydrate);

  useEffect(() => {{
    const handler = (e: StorageEvent) => {{
      if (e.key === 'dzpharm-store' && e.newValue) {{
        // Re-hydrate the Zustand store from the updated localStorage
        rehydrate();
      }}
    }};

    window.addEventListener('storage', handler);
    return () => window.removeEventListener('storage', handler);
  }}, [rehydrate]);
}}

// Usage in the root layout
export default function RootLayout({{ children }}) {{
  useCrossTabSync();
  return <html><body>{{children}}</body></html>;
}}
```

7.4 The Grade Impact

C+

CURRENT (6 GAPS)

B+

AFTER SAME-DAY THREE (1 DAY)

A-

AFTER #1 ROUTING (2 WEEKS)

A

AFTER #8 AUTH SERVER-SIDE (4 WEEKS)

A+

AFTER ALL 6 GAPS CLOSED

~150

LINES OF CODE (SAME-DAY THREE)

THE SAME-DAY THREE · HIGHEST-ROI FIXES

Three fixes, one day, ~150 lines of code, no architectural changes. The drug list in URL params (4h) closes the most expensive state-loss on the platform. The Copilot history in Zustand (2h) closes the most frustrating state-loss. The cross-tab sync (2h) closes the most subtle state-loss. Ship these three as a single PR in week 1 — they're the highest-ROI state-management fixes in the entire review menu.

CHAPTER 08 · CROSS-TAB SYNC & SERVER-SIDE STATE

From Per-Browser to Per-User

The current Zustand store is per-browser (localStorage). Favorites, Copilot history, and recently-viewed sync within a single browser but not across devices. The cross-tab sync

(Fix #3) closes the within-browser gap. The server-side state (after #8 Auth) closes the cross-device gap.

8.1 The Three Layers of State

LAYER	WHERE	SURVIVES	SYNCS	STATUS
URL state	Browser address bar	Refresh, share, bookmark	Per-tab (no sync needed)	AFTER #1
Client state	Zustand + localStorage	Refresh, tab close (persist)	Cross-tab (via storage event)	AFTER FIX #3
Server state	Prisma + PostgreSQL	Everything (including device switch)	Cross-device	AFTER #8

8.2 The Cross-Tab Sync Pattern

The `storage` event fires in Tab B when Tab A modifies `localStorage`. By listening to this event and calling Zustand's `rehydrate()`, Tab B picks up Tab A's changes immediately. This is the simplest cross-tab sync pattern — no WebSocket, no BroadcastChannel API, no server. Effort: 2 hours (Fix #3, covered in Chapter 7.3).

8.3 The BroadcastChannel Alternative

```
// Alternative - BroadcastChannel API (more powerful, slightly more code)
const channel = new BroadcastChannel('dzpharm-sync');

// Tab A: broadcast changes
useThemeStore.subscribe((state) => {{
  channel.postMessage({{ type: 'state-update', state }});
}});

// Tab B: listen for changes
channel.onmessage = (e) => {{
  if (e.data.type === 'state-update') {{
    useThemeStore.setState(e.data.state);
  }}
}};
```

The BroadcastChannel API is more powerful than the storage event (it can carry structured data, not just strings) but has slightly less browser support (not in IE11, but DzPharm doesn't target IE). For DzPharm's use case, the storage event is sufficient. BroadcastChannel is the alternative if more complex cross-tab communication is needed later.

8.4 The Server-Side Migration (After #8 Auth)

Once per-user accounts exist (#8 Auth), the client-side Zustand store can be augmented with a server-side store (Prisma + PostgreSQL). The pattern is: (1) on login, fetch the user's server-side state (favorites, Copilot conversations, recently-viewed), (2) merge it with the local Zustand state, (3) on every

state change, debounce-sync to the server (300ms delay to batch rapid changes). This gives the user the same state on every device — work, home, mobile.

```
// The server-side sync pattern (after #8 Auth)
import {{ useEffect, useRef }} from 'react';
import {{ useSession }} from 'next-auth/react';

export function useServerSideSync() {{
  const {{ data: session }} = useSession();
  const debounceRef = useRef(setTimeout(() => {{}}, 0));

  // On login: fetch server-side state and merge
  useEffect(() => {{
    if (session?.user?.id) {{
      fetchServerState(session.user.id).then((serverState) => {{
        useThemeStore.getState().mergeServerState(serverState);
      }});
    }}
  }}, [session?.user?.id]);

  // On state change: debounce-sync to server (300ms)
  useEffect(() => {{
    const unsubscribe = useThemeStore.subscribe((state) => {{
      clearTimeout(debounceRef.current);
      debounceRef.current = setTimeout(() => {{
        syncToServer(session.user.id, state);
      }}, 300);
    }});
    return unsubscribe;
  }}, [session?.user?.id]);
}}
```

8.5 The Optimistic Update Pattern

The server-side sync uses optimistic updates: the Zustand store updates immediately (the UI reflects the change instantly), and the server sync happens in the background (300ms debounce). If the server sync fails (network error), the store keeps the local state and retries. This is the correct pattern for responsive client-side apps with server-side persistence — the user never waits for the server.

— CHAPTER 09 • TESTING & MIGRATION STRATEGY

State Verification & Safe Upgrades

State management changes touch every feature that reads from the Zustand store. The testing strategy must verify: (1) state persists across refresh, (2) state syncs across tabs, (3) the schema migration from v2 to v3 works correctly, (4) the store prune logic doesn't lose data, (5) the URL state and Zustand state don't conflict.

9.1 The State Test Matrix

STATE	PERSIST?	CROSS-TAB?	SHARE?	SERVER-SIDE?
Favorites	AFTER FIX #3	AFTER FIX #3	No	AFTER #8
Copilot history	AFTER FIX #2	AFTER FIX #3	No	AFTER #8
Recently viewed	AFTER FIX #5	AFTER FIX #3	No	AFTER #8
Interactions list	AFTER FIX #1	N/A	AFTER FIX #1	No (URL is enough)
Theme + palette	YES (EXISTING)	AFTER FIX #3	No	No (client-only)

9.2 The Migration Test

```
// Test the v2 → v3 schema migration
describe('Zustand store migration v2 → v3', () => {{
  it('adds copilotMessages and copilotMode to v2 state', () => {{
    const v2State = {{
      state: {{ favorites: [], recentlyViewed: [], palette: 'craie-cobalt' }},
      version: 2,
    }};

    const migrated = migrate(v2State, 2);

    expect(migrated.state.copilotMessages).toEqual([]);
    expect(migrated.state.copilotMode).toBe('pro');
    expect(migrated.state.favorites).toEqual([]); // Preserved
    expect(migrated.version).toBe(3);
  }});

  it('preserves existing favorites during migration', () => {{
    const v2State = {{
      state: {{ favorites: [{ id: 9775, brand: 'DOLIPRANE' }] }},
      version: 2,
    }};

    const migrated = migrate(v2State, 2);

    expect(migrated.state.favorites).toHaveLength(1);
    expect(migrated.state.favorites[0].id).toBe(9775);
  }});
}});
```

9.3 The Cross-Tab Sync Test

```
describe('cross-tab sync', () => {{
  it('syncs favorites across tabs via storage event', () => {{
    // Tab A: add a favorite
    useThemeStore.getState().addFavorite({{ id: 9775, brand: 'DOLIPRANE' }});

    // Simulate storage event (Tab B picks it up)
    const newState = JSON.parse(localStorage.getItem('dzpharm-store')!);
    window.dispatchEvent(new StorageEvent('storage', {{
      key: 'dzpharm-store',
      newValue: JSON.stringify(newState),
    }}));

    // Tab B: favorite should be visible
    const favorites = useThemeStore.getState().favorites;
    expect(favorites).toHaveLength(1);
    expect(favorites[0].id).toBe(9775);
  }});
});
```

— CHAPTER 10 • FINAL ROADMAP & VERDICT

The 2-Week Plan to Go From C+ to A+

The State Management upgrade is the platform's **third foundation** — after #1 Routing (URLs) and #8 Auth (identity). In 2 weeks (plus dependencies on #1 and #8), it transforms the platform from C+ grade (6 state gaps, features losing data on refresh) to A+ grade (persistent, URL-backed, cross-tab synced, server-side after auth).

10.1 The Grade Trajectory

C+

CURRENT (6 GAPS)

B+

AFTER SAME-DAY THREE (1 DAY)

A-

AFTER #1 ROUTING (2 WEEKS)

A

AFTER GAP #5 FIX (1 HOUR)

A+

AFTER #8 AUTH SERVER-SIDE (4 WEEKS)

~150

LINES OF CODE (DAY 1)

10.2 The Phased Plan

PHASE	DURATION	FIXES	CLOSES GAPS	GRADE AFTER
Phase 1 · Same-Day Three	1 day	URL drug list + Zustand Copilot + cross-tab sync	#3, #4, #6 (partial)	B+
Phase 2 · Recently-viewed cap	1 hour	Fix dedup bug, raise cap to 10	#5	A-
Phase 3 · Routing migration	2 weeks (after #1)	Active view → URL, medication detail → intercepting route	#1, #2	A
Phase 4 · Server-side state	4 weeks (after #8)	Favorites, Copilot, recently-viewed → Prisma + PostgreSQL	#6 (full), cross-device	A+

10.3 The ROI Summary

INVESTMENT	RETURNS
1 day (Same-Day Three)	3 state gaps closed, ~150 lines of code, no architectural changes
1 hour (recently-viewed cap)	Gap #5 closed, recently-viewed becomes useful (10 items vs 1)
2 weeks (after #1 Routing)	Gaps #1 + #2 closed, active view in URL, medication detail shareable
4 weeks (after #8 Auth)	Server-side state, cross-device sync, all 6 gaps fully closed
Total: ~2 weeks (own work) + dependencies	C+ → A+, 6 gaps closed, 8 features improved

10.4 The Five Pillars Together

The five deep-dive technical design documents form the platform's **complete transformation blueprint**:

- ✗ #1 Routing (2 weeks) — the URL foundation. 9 features unlocked, 380× SEO. D → A+.
- ✗ #8 Auth (4 weeks) — the identity foundation. 6 features unlocked, P0 closed, compliance. D → A+.
- ✓ #2 State Management (2 weeks) — the state foundation. 6 gaps closed, 8 features improved. C+ → A+.
- ✓ #36 Copilot (2 weeks) — the engagement foundation. 5 bugs fixed, 10× latency. B → A+.
- ✓ #53 Color System (2 weeks) — the design foundation. Governance + verification. A → A+.

Together: **~12 weeks of engineering**. Three P0 foundations fixed (#1 + #8 + #2), the signature feature elevated (#36), and the crown jewel protected (#53). The platform goes from «impressive demo with critical gaps» to «production-grade clinical infrastructure with 190K monthly organic visitors, regulatory compliance, persistent state across devices, a streaming AI assistant, and a world-class design system».

FINAL VERDICT · THE STATE FOUNDATION

The State Management upgrade is the platform's **state foundation**. It transforms the platform from C+ grade (6 state gaps, features losing data on refresh) to A+ grade (persistent, URL-backed, cross-tab synced, server-side after auth). The «same-day three» fixes (1 day, ~150 lines of code) close 3 of the 6 gaps with no architectural changes. The remaining 3 gaps close via #1 Routing (gaps #1, #2) and #8 Auth (gap #6 full + server-side). Together with #1, #8, #36, and #53, this completes the five-pillar platform transformation.

Immediate Action · Ship the Same-Day Three Today

1. Create `lib/useInteractionListState.ts` hook (drug list in URL params) — 4 hours.
2. Add `copilotMessages` + `copilotMode` slices to the Zustand store — 2 hours.
3. Add `useCrossTabSync` hook (storage event listener) — 2 hours.
4. Fix the recently-viewed cap (raise from 1 to 10, fix dedup bug) — 1 hour.
5. Bump store version to 3, add the migrate function — 30 minutes.
6. Test: refresh, cross-tab, migration — 2 hours.
7. Deploy. Grade lifts from C+ to B+.

A+

From C+ (6 state gaps, features losing data on refresh) to A+ (persistent, URL-backed, cross-tab synced, server-side after auth). The same-day three fixes close 3 gaps in 1 day with ~150 lines of code.

10

CHAPTERS

9

SCREENSHOTS

6

GAPS CLOSED

8

FEATURES IMPROVED